

Candidates Feature — Real Data Schema & Design Document

Project: HR Dashboard — Candidates Module **Status:** Schema finalized, ready for implementation **Date:** 2026-08-27

1. Purpose

The HR Dashboard's Candidates section currently displays **dummy/mock data** (hardcoded in `main/frontend/src/data/hrMockData.js`). This document defines the finalized data schema and design decisions required to convert this into a **real, Firebase-backed feature** that is:

- **Fast** — minimal read latency, no unnecessary round-trips
- **Fresh** — data reflects real-time or near-real-time state
- **Quota-safe** — stays within Firebase's free/paid read-quota limits by avoiding the anti-patterns already identified in this project (unfiltered global listeners, unpaginated scans, N+1 queries, double-read-per-write)

The design choices below were made collaboratively, with each decision weighed against the perspective of an experienced HR professional (what they actually need day-to-day) and against Firebase cost/performance constraints.

2. Background: What Exists Today

The backend (`main/backend/controllers/hrDeskController.js`) already has a `candidates` Firestore collection wired through a generic `makeCrud` factory — the same factory used for `employees` . The frontend mock data (`hrMockData.js`) defines the shape that this collection was originally modeled on, plus two related mock arrays: `interviews` and `feedbackEntries` .

The `HrDeskContext` (`main/frontend/src/context/HrDeskContext.jsx`) is the shared data layer already used for `employees` , `candidates` , `interviews` , and `attendanceRecords` — it fetches all four resources once in a single `Promise.all` , gated by role (`canSeeHrDesk : hr , founder` only), and exposes them via context so pages don't each run their own fetch. Any new candidates implementation should plug into this existing pattern rather than introducing a new one.

3. Candidate Document Schema (Finalized)

Field	Type	Description
<code>id</code>	Firestore auto ID	Unique document identifier
<code>name</code>	string	Candidate's full name
<code>email</code>	string	Contact email
<code>phone</code>	string	Contact phone number
<code>location</code>	string	Candidate's current city/location
<code>skills</code>	array of strings	Skill tags (e.g. ['React' , 'Node.js'])

experience	number	Years of experience (changed from free-text like "4 yrs" to a plain number, so it can be filtered/sorted — e.g. "show candidates with 3–5 years experience")
education	string	Highest qualification / institute
currentCTC	number (new)	Candidate's current salary
expectedSalary	number	Expected salary (changed from free-text like "₹18 LPA" to a number for the same filtering/sorting reason)
currentCompany	string	Current employer
noticePeriod	string/number (new)	Notice period at current job (exact unit — days vs. free text — to be finalized during implementation)
portfolio	string	Portfolio/personal site URL (can be empty string)
source	enum string	Where the application came from (see §4)
stage	enum string	Current pipeline stage (see §4)
appliedFor	string	Job title/position applied for
appliedOn	Firestore Timestamp	Application date (changed from a plain 'yyyy-mm-dd' string to a proper Timestamp, for correct sorting and pagination)
resumeFileName	string	Original resume filename (see §5)
resumeUrl	string (new)	Firebase Storage download URL for the resume (see §5)
rejectionReason	enum string (new)	Reason the candidate was rejected or declined an offer (see §6)
assignedRecruiter	string (new)	The HR person who owns/is responsible for this candidate
lastUpdatedBy	string (new)	Audit field — records which user made the most recent edit
nextInterview	object (new)	Denormalized summary: { date, type, interviewer } for the candidate's next scheduled interview (see §7)
created_at	ISO string (auto)	Set automatically on document creation (existing backend convention)
updated_at	ISO string (auto)	Set automatically on every update (existing backend convention)

Why these changes from the original mock data?

- `experience` and `expectedSalary` were free-text strings ("4 yrs" , "₹18 LPA") in the mock data. This makes filtering and reporting (e.g. average expected salary, experience-range filters) impossible without error-prone string parsing. Converting them to numbers fixes this at the schema level.
- `appliedOn` was a plain string date. A Firestore Timestamp allows correct chronological sorting and range queries (e.g. "candidates applied in the last 30 days") directly at the database level, which is both faster and cheaper than fetching everything and sorting client-side.

4. Stage & Source Enumerations (Finalized)

4.1 Candidate Stage (CANDIDATE_STAGES)

The pipeline stage a candidate is currently in:

1. Applied
2. Screening
3. HR Round
4. Technical Round
5. Final Interview
6. Offer Sent
7. Joined
8. Offer Declined (*new*)
9. Rejected
10. On Hold (*new* — *not sequential; can apply at any point in the pipeline as a pause status, not a terminal outcome*)

Rationale for additions:

- **Offer Declined** — previously, a candidate declining an offer was indistinguishable from the company rejecting the candidate (both fell under "Rejected"). Separating them allows HR to track "how many accepted offers are declined by candidates" as its own metric — a materially different signal from a company-side rejection.
- **On Hold** — covers cases where a candidate's process is paused (e.g. position frozen, budget on hold) rather than permanently closed. Without this, HR previously had no way to represent "paused" without incorrectly marking someone as rejected.

4.2 Application Source (RESUME_SOURCES)

Where the candidate's application originated:

1. LinkedIn
2. Naukri
3. Indeed
4. Monster
5. Internal Portal
6. Referral
7. Manual Upload
8. Company Website / Career Page (*new*)

Rationale for addition: Anticipates a future public application form on the company's careers page — without this value, such applications would have no accurate source to record.

5. Resume Storage (Finalized)

Approach: Store both a filename and a Storage URL.

- On upload, the resume is stored in Firebase Storage, and its download URL is captured **once, at upload time** and saved into the candidate document as `resumeUrl` (alongside `resumeFileName`).
- **Why not fetch the URL on-demand instead?** If HR opens a candidate's resume multiple times (which happens frequently during screening), generating the URL fresh each time means an extra Firebase Storage

call per view. Saving it once at upload time turns a repeated cost into a one-time cost — directly supporting the "quota-safe" goal.

Additional rules:

- **File type:** PDF only (no Word or other formats accepted).
- **File naming:** the original filename plus an appended serial number, to prevent name collisions in Storage when multiple candidates (or the same candidate re-uploading) use similar filenames.
- **Replace policy:** when a candidate uploads a new resume, the previous file is deleted from Storage. No version history is kept — only the current resume exists at any time.

6. Rejection Reason (Finalized)

Approach: Dropdown/enum, not free text.

A fixed, structured list of reasons allows HR to run meaningful reports later (e.g. "what is the #1 reason candidates are lost at this stage") without needing to parse inconsistent free-text entries.

Proposed values (exact final wording open to refinement during implementation):

- Salary Mismatch
- Skill Gap
- Culture Fit
- Position Closed/On Hold
- Candidate Withdrew
- Better Candidate Selected
- Notice Period Too Long
- Other

Applies to both: the `Rejected` stage **and** the `Offer Declined` stage — both outcomes benefit from a structured reason, since "why do candidates decline our offers" is just as valuable a report as "why do we reject candidates."

7. Interview & Feedback Data Structure (Finalized — Hybrid Approach)

Interviews and feedback have their own operational needs beyond just being "attached to a candidate" — for example, an interviewer needs to see their own schedule across *all* candidates, and HR may want a "today's interviews" view across the whole pipeline. This ruled out simply embedding interview arrays inside each candidate document, since Firestore cannot efficiently query into nested arrays across many documents for such cross-candidate views.

Decision: Hybrid structure.

1. **Full interview and feedback data** remain in their own top-level Firestore collections — `interviews` and `feedbackEntries` — each linked back to a candidate via a `candidateId` field. This preserves the ability to query across all candidates (e.g., "all interviews scheduled today," "all feedback given by interviewer X").
2. **The candidate document additionally carries a small denormalized summary field**, `nextInterview`: `{ date, type, interviewer }`, representing only the *next upcoming* interview for that candidate. This field is updated whenever an interview is scheduled or rescheduled for that candidate.
3. **Full interview history and past feedback are only fetched when viewing a candidate's individual detail page** — not in the main candidates list view.

Why this matters for performance: Without the denormalized `nextInterview` field, showing "next interview date" in the candidates list would require one extra query *per candidate* — a classic N+1 pattern that multiplies Firestore reads linearly with the number of candidates shown. The summary field collapses this to a single field already present on the document being read anyway, at the cost of a small amount of duplicated data that is easy to keep in sync (updated only when an interview is scheduled/rescheduled — an infrequent event).

8. Ownership & Audit Fields (Finalized)

Two distinct fields, serving two distinct purposes:

- **assignedRecruiter** — the HR person responsible for/owning this candidate. Set manually when a candidate is added to the system or reassigned to a different recruiter. This answers "who is driving this candidate forward?"
- **lastUpdatedBy** — an audit field, set automatically on every edit (alongside the existing `updated_at` timestamp). This answers "who last touched this record?" — useful for accountability when multiple HR users work the same pipeline.

Both fields are kept, rather than picking just one, because ownership and audit trail answer different questions and neither can substitute for the other.

9. Performance & Firebase Quota Considerations

This schema and its access pattern are designed in line with the project's broader Firebase quota remediation plan. The specific practices that apply to the Candidates feature:

- **Single shared listener/context** — candidates should be fetched once via the existing `HrDeskContext`, not independently by every component that needs candidate data.
 - **Role-gating** — only users with `hr` or `founder` roles should trigger a candidates fetch at all; other roles should never attach a listener to this collection.
 - **Pagination** — the candidates list should load in pages (e.g. 20–25 at a time) rather than fetching the entire collection at once, with a sensible default filter (e.g. active/open candidates) so old closed records aren't loaded by default.
 - **Server-side filtering** — status/position/source filters should be applied as Firestore query clauses (`where( . . . )`), not fetched in bulk and filtered in JavaScript afterward.
 - **Real-time vs. one-time reads** — only the views where "freshness" genuinely matters (e.g. the live pipeline board) should use a real-time listener (`onSnapshot`); reporting/export views should use a one-time read (`getDocs`).
 - **No double-read-per-write** — after updating a candidate's stage or details, the UI should update from local state (optimistic update) rather than immediately re-fetching the document from Firestore.
 - **Denormalization over N+1** — as detailed in §7, summary data needed for list views (like `nextInterview`) is stored directly on the candidate document rather than requiring a join-like extra query per row.
-

10. Summary of All Decisions

Area	Decision
experience / expectedSalary	Numbers, not free text
appliedOn	Firestore Timestamp, not a string

currentCTC, noticePeriod	Added as new fields
Candidate stages	Original 8 + Offer Declined + On Hold (10 total)
Application sources	Original 7 + Company Website/Career Page (8 total)
Resume storage	Filename + Storage URL saved at upload time; PDF only; original name + serial number; old file deleted on replace
Rejection reason	Dropdown/enum; applies to both Rejected and Offer Declined
Interviews/feedback	Hybrid — separate collections for full data, nextInterview summary denormalized onto the candidate document
Ownership/audit	Both assignedRecruiter and lastUpdatedBy kept, for different purposes

Status: Schema is fully finalized. No fields remain undecided. The next step is implementation — updating the backend `makeCrud` field list for the `candidates` collection and wiring the frontend through the existing `HrDeskContext` , following the quota-safe practices in §9.